Assignment 1

Fabiano Blasch, Nik Gabrovsek, Patrick Gyasi, Gregor Steiner

May, 2021

Time Series Analysis

1

We can use the **quantmod** package to download the data from the FRED database.

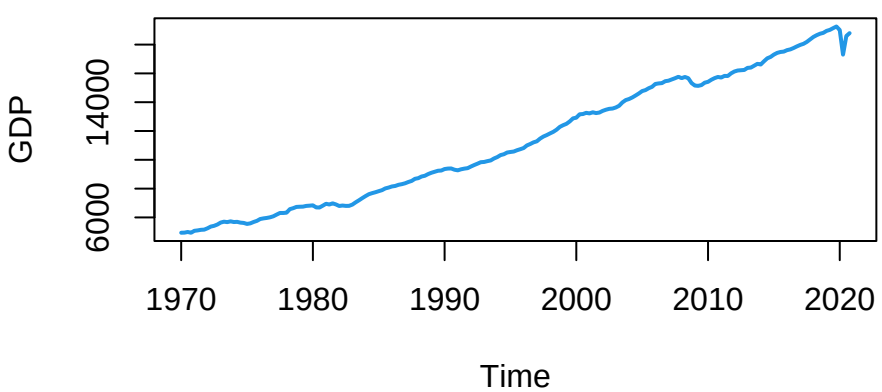
```
# Download data
quantmod::getSymbols("GDPC1", src = "FRED")
```

```
## [1] "GDPC1"
```

```
gdp.ts <- ts(GDPC1, start = c(1947, 1), frequency = 4)

# Filter
gdp.ts <- window(gdp.ts, start = c(1970, 1), end = c(2020, 4))
```

After filtering for the time period we are interested in, we can now plot the series.

```
# Plot
plot(gdp.ts, lwd = 2, col = 4, ylab = "GDP")
```



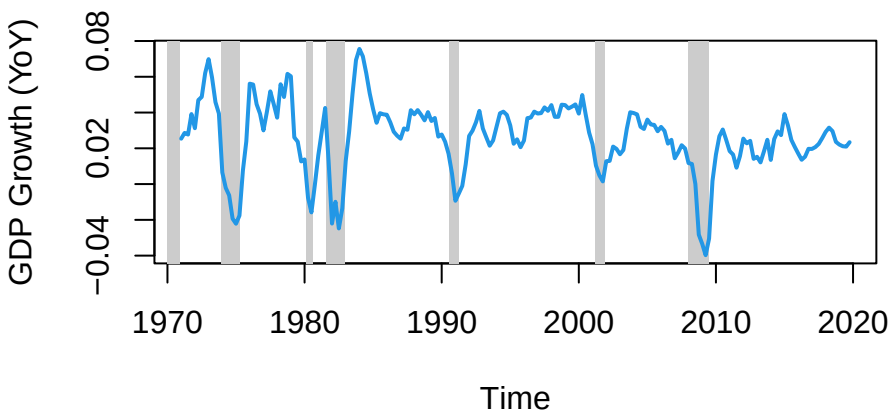
2

Trend is a slow change that occurs over a long period of time. In our case trend is clearly positive. Cyclical component is movement around our trend line, its of short time nature.The cycle component is closely related to business cycles. Time series with trend in it are normally not stationary. Stationary time series have no

predictable patterns in long term which is clearly not the case for GDP. Main idea of detrending is to get rid of a trend in a time series. Detrending changes our un-stationary time series into a stationary one. We know different ways of detrending. First one I will mention is usage of growth rates, we basically take a difference of logarithm between two data points. This gives us a growth rate time series which is stationary. Next one I will mention is linear/nonlinear detrending, if we assume our trend is linear, we can estimate linear regression of our time-series this will give us a trend and we can estimate our detrended time-series with observation of deviations from linear regression line. Lastly we can use filters to detrend data, there are various different filter such as HP and Bandpass filter. They are similar to linear detrending, but we can have smoother line of best-fit as trend line is not necessarily linear.

3

```r
# Compute y/y growth rates
gdp.growth <- diff(gdp.ts, 4) / lag(gdp.ts, 4)

# Plot
plot(gdp.growth, type = "n", ylab = "GDP Growth (YoY)")
tis::nberShade()
lines(gdp.growth, lwd = 2, col = 4)
```



4

```r
# Test for stationarity
tseries::adf.test(gdp.growth)
```

```
## Warning in tseries::adf.test(gdp.growth): p-value smaller than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  gdp.growth
## Dickey-Fuller = -4.5746, Lag order = 5, p-value = 0.01
## alternative hypothesis: stationary
```

We do reject the null-hypothesis of non-stationarity. Thus, the series is stationary.

5

We want to estimate the model

$$y_t = c + \varphi_1 y_{t-1} + \varphi_2 y_{t-2} + \varepsilon_t$$

in a Bayesian fashion. In matrix notation this reads as

$$y = X\varphi + \varepsilon,$$

where $y := [y_3, ..., y_T]', \varphi := [c, \varphi_1, \varphi_2]', \varepsilon := [\varepsilon_3, ..., \varepsilon_T]'$ and

$$X := \begin{bmatrix} 1 & y_2 & y_1 \\ \vdots & \vdots & \vdots \\ 1 & y_{T-1} & y_{T-2} \end{bmatrix}.$$

We obviously lose the first two observations, leaving us with $T-2$ observations.

The first step is choosing prior distributions for c, φ_1, φ_2. A simple normal prior might be a good start. Thus,

$$\varphi \sim N_3(\mu_0, \Sigma_0),$$

where $\mu_0 = [\mu_c, \mu_{\varphi_1}, \mu_{\varphi_2}]'$ and $\Sigma_0 = diag(\sigma_c^2, \sigma_{\varphi_1}^2, \sigma_{\varphi_2}^2)$.

Then, we only need a prior distribution for the error variance, where we will use an Inverse Gamma Distribution as in the linear model:

$$\sigma^2 \sim IG(c_0, d_0).$$

And we know that

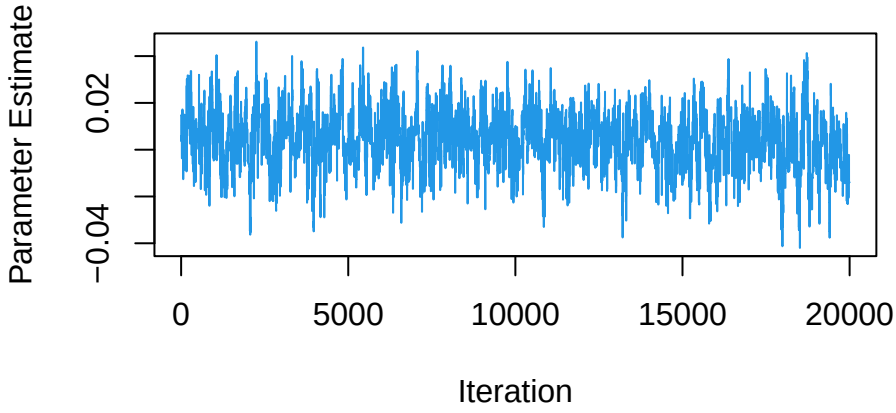
$$y|\varphi, \sigma^2 \sim N_{T-2}(X\varphi, \sigma^2 I_{T-2}).$$

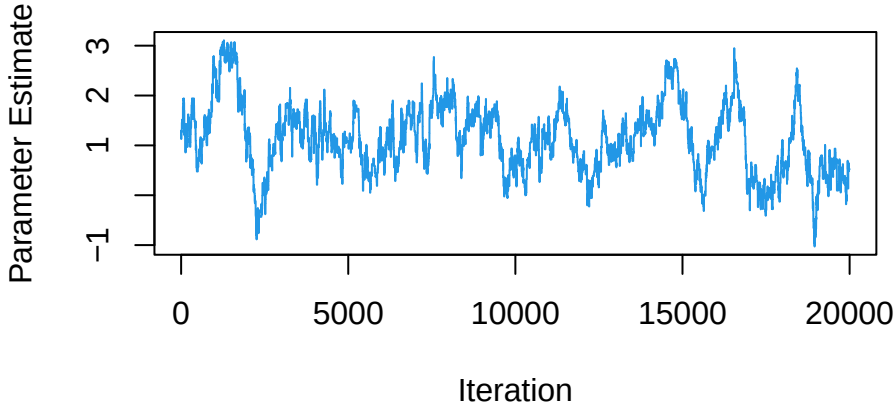
Now, we can start setting up the Metropolis algorithm to estimate the posterior density.

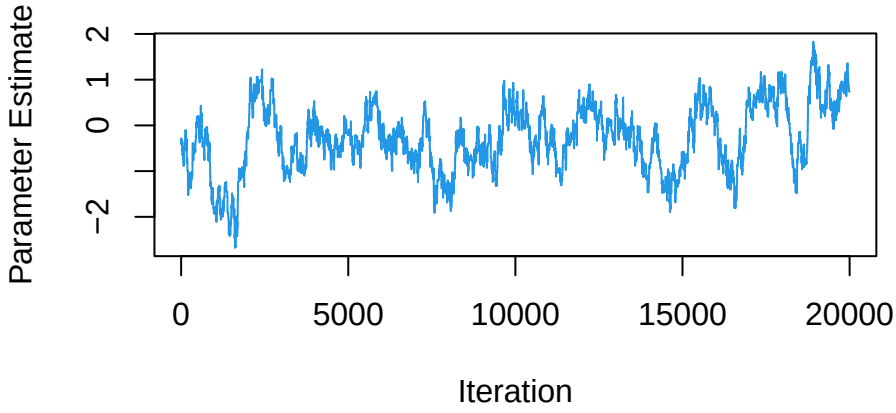
```r
########## Metropolis Algorithm ##########


# Set up X matrix and y vector
T <- length(gdp.growth)
X <- cbind(rep(1, T-2), gdp.growth[-c(1, T)], gdp.growth[-c(T-1, T)])
y <- gdp.growth[-c(1, 2)]


# Prior Setup

# Normal prior on phi vector
mu0 <- c(0, 0, 0)
sig2.0 <- 100 # Same variance for the three parameters

# IG prior on sigma2
c0 <- 2
d0 <- 1

# Algorithm Setup
nburn <- 2e4
nsave <- 2e4
ntot <- nburn + nsave
t <- nrow(X)
```

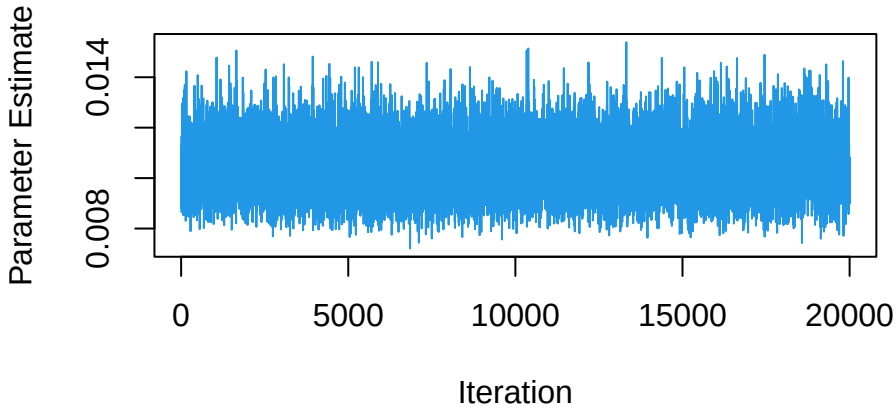
```r
# proposal draw variances
eta2.c <- 0.05
eta2.phi1 <- 0.4
eta2.phi2 <- 0.4
eta2 <- c(eta2.c, eta2.phi1, eta2.phi2)

# starting values
sig2.draw <- 1
phi.draw  <- c(0, 0, 0)
accept    <- 0


# Storage matrix
phi.store <- matrix(NA, nsave, 3)
colnames(phi.store) <- c("c", "phi1", "phi2")

sig.store <- matrix(NA, nsave, 1)


# Metropolis algorithm loop
for (i in 1:ntot) {

  ## STEP 1:

  # Create proposal from normal distribution
  #phi.prop <- MASS::mvrnorm(1, phi.draw, diag(sqrt(eta2)))
  phi.prop <- mapply(function(x, y) rnorm(1, x, sd = sqrt(y)), phi.draw, eta2)

  # Evaluate prior for proposal
  prior.current <- sum(dnorm(phi.draw, mu0, sqrt(sig2.0), log = TRUE))
  prior.prop    <- sum(dnorm(phi.prop, mu0, sqrt(sig2.0), log = TRUE))

  # Evaluate LL
  ll.current <- sum(dnorm(y, X %*% phi.draw, sqrt(sig2.draw), log = TRUE))
  ll.prop <- sum(dnorm(y, X %*% phi.prop, sqrt(sig2.draw), log = TRUE))

  # And add prior
  c.p.current <- ll.current + prior.current
  c.p.prop <- ll.prop + prior.prop

  # And LR
  ll.ratio <- c.p.prop - c.p.current

  if(ll.ratio > log(runif(1))){ #if our proposal is likely "enough"

      #replace the current draw for phi with our proposal
      phi.draw <- phi.prop

      #add 1 to our acceptance counter
      accept   <- accept + 1

  }
```

```r
  # Tune proposal variance during first 50% of burn-in
  if (i < 0.5 * nburn) {
    if(accept / i > 0.3) eta2 <- eta2 * 1.01
    if(accept / i < 0.15) eta2 <- eta2 * 0.99
  }

  ## STEP 2: SAMPLE SIGMA2 GIVEN PHI & DATA

  # compute errors
  e  <- y - X %*% phi.draw

  # sum of squared residuals (equivalent to sum((e)^2))
  ee <- t(e) %*% e

  # compute posterior quantities
  ck <- c0 +  t/2
  dk <- d0 + ee/2
  # sample sigma2 from inverse gamma posterior
  sig2.draw <- 1 / rgamma(1, ck, dk)

  # STEP 3: STORE POSTERIOR SAMPLES

  #only after burn-in period!
  if(i > nburn){

  phi.store[i - nburn,]  <- phi.draw
  sig.store[i - nburn,] <- sig2.draw

  }


}


# Plot traceplots
apply(cbind(phi.store, sig.store), 2, function(x) plot.ts(x , xlab = "Iteration", ylab= "Parameter Estim
```


Parameter Estimate
0.02
−0.04
0
5000
10000
15000
20000
Iteration

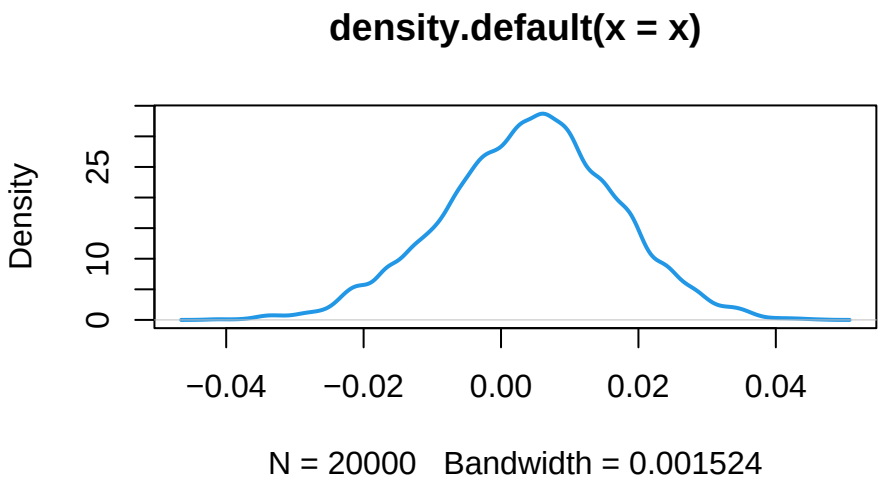

Parameter Estimate
3
2
1
−1
0
5000
10000
15000
20000
Iteration

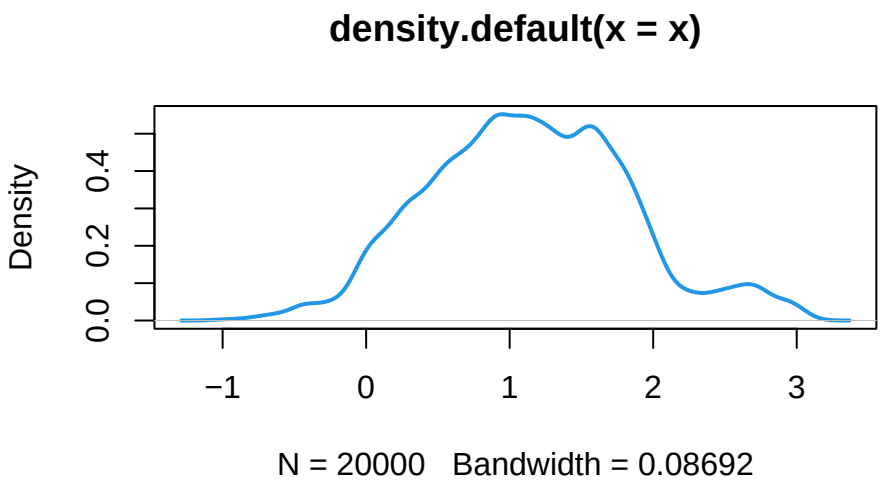


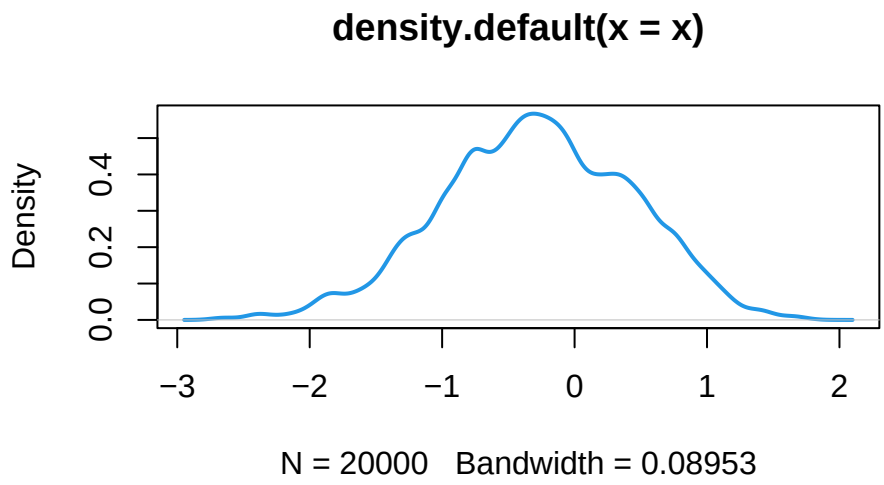


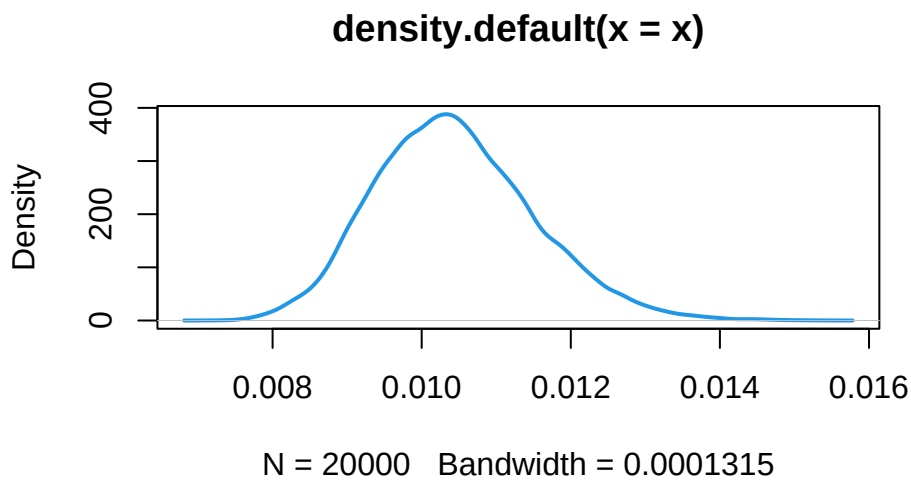

```
## NULL
```

```
# Plot densities
apply(cbind(phi.store, sig.store), 2, function(x) plot(density(x), lwd = 2, col = 4))
```


density.default(x = x)
Density
0
10
25
−0.04
−0.02
0.00
0.02
0.04
N = 20000 Bandwidth = 0.001524


density.default(x = x)
Density
0.0
0.2
0.4
−1
0
1
2
3
N = 20000 Bandwidth = 0.08692






```
## NULL
```

Let's have a look at the posterior distribution the algorithm ended up with.

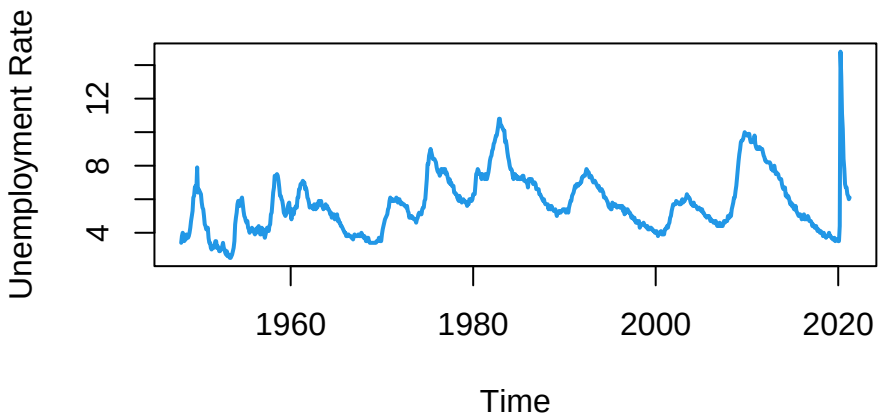
Prior Sensitivity

Again, we can use the **quantmod** package to download the data from the FRED database.

```r
# Download data
quantmod::getSymbols("UNRATE", src = "FRED")
```

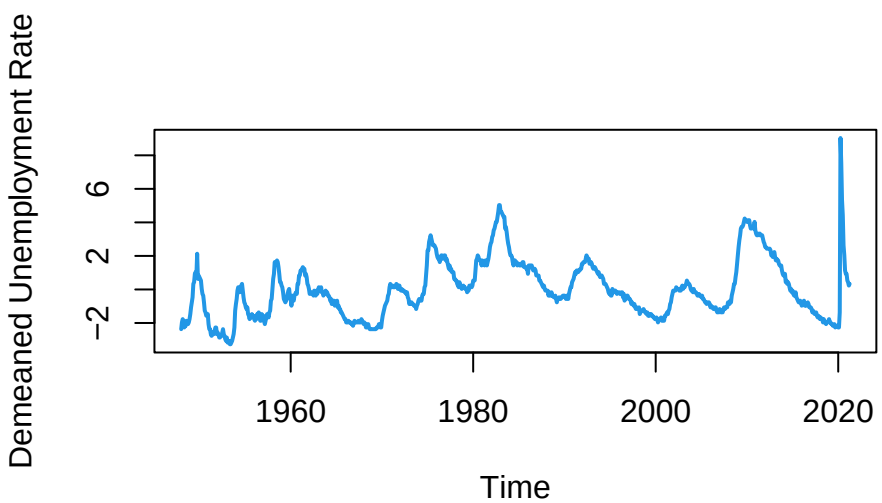
```
## [1] "UNRATE"
```

```r
un.ts <- ts(UNRATE, start = c(1948, 1), frequency = 12)

# Plot
plot(un.ts, col = 4, lwd = 2, ylab = "Unemployment Rate")
```



1

```
# Demean series
un.demeaned <- un.ts - mean(un.ts)

# Plot
plot(un.demeaned, col = 4, lwd = 2, ylab = "Demeaned Unemployment Rate")
```



2

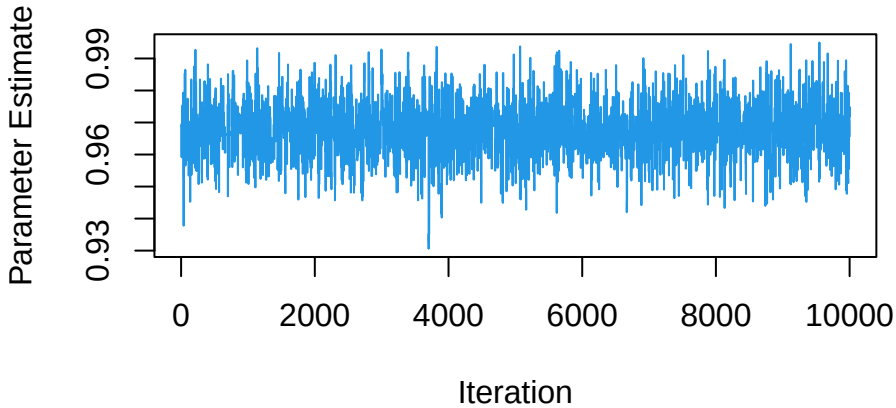
We will use the Metropolis algorithm used in the lecture.

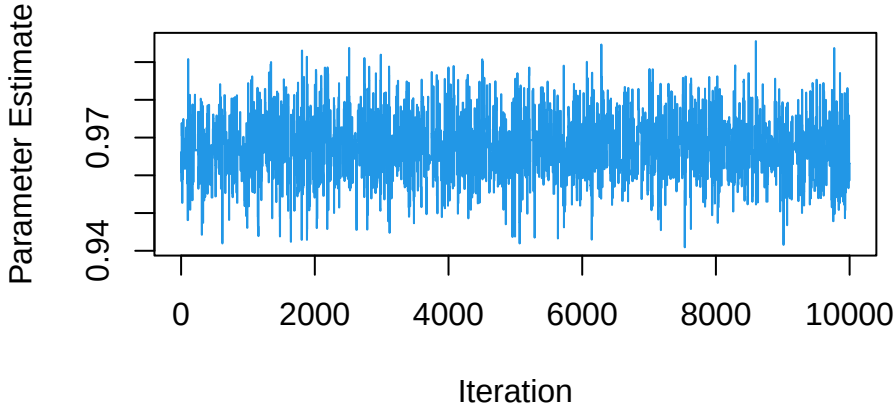
```
metro.est <- function(y, x, prior, prior.params){


  # IG prior on sigma2
```

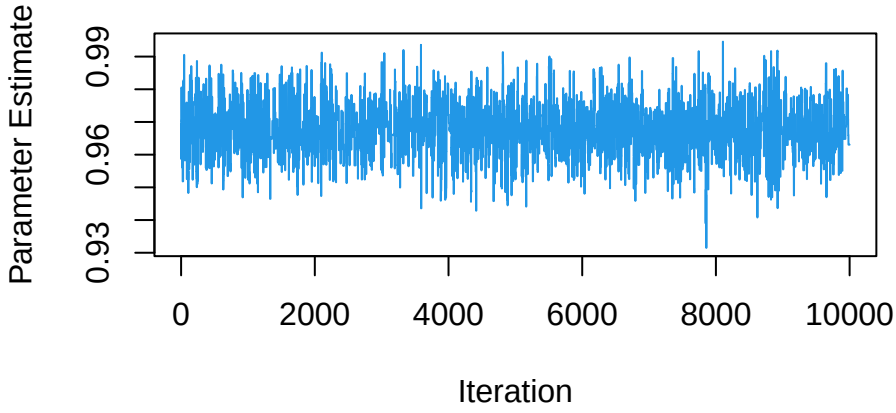
```r
c0 <- 2
d0 <- 1

# Algorithm Setup
nburn <- 1e4
nsave <- 1e4
ntot <- nburn + nsave
t <- length(x)
eta2  <- 0.0161

sig2.draw <- 1
phi.draw  <- 0.5
accept    <- 0


# Storage matrix
phi.store <- matrix(NA, nsave, 1)

sig.store <- matrix(NA, nsave, 1)


# Metropolis algorithm loop
for (i in 1:ntot) {

  ## STEP 1:

  # Create proposal
  phi.prop <- rnorm(1, phi.draw, sqrt(eta2))


  # Evaluate prior for proposal
  prior.current <- do.call(paste0("d", prior), list(phi.draw, prior.params[1], prior.params[2]))
  prior.prop    <- do.call(paste0("d", prior), list(phi.prop, prior.params[1], prior.params[2]))

  # Evaluate LL
  ll.current <- sum(dnorm(y, x * phi.draw, sqrt(sig2.draw), log = TRUE))
  ll.prop <- sum(dnorm(y, x * phi.prop, sqrt(sig2.draw), log = TRUE))

  # And add prior
  c.p.current <- ll.current + prior.current
  c.p.prop <- ll.prop + prior.prop

  # And LR
  ll.ratio <- c.p.prop - c.p.current

  if(ll.ratio > log(runif(1))){ #if our proposal is likely "enough"

      #replace the current draw for rho with our proposal
      phi.draw <- phi.prop

      #add 1 to our acceptance counter
      accept   <- accept + 1
```

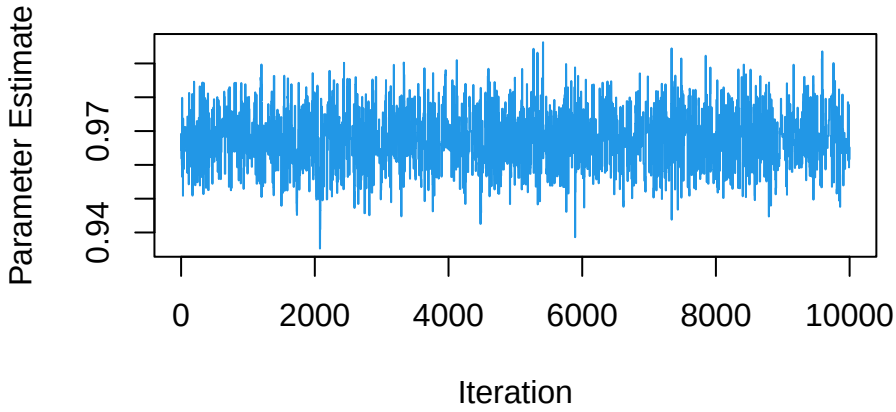
```r
      }

    # Tune proposal variance during first 50% of burn-in
    if (i < 0.5 * nburn) {
      if(accept / i > 0.3) eta2 <- eta2 * 1.01
      if(accept / i < 0.15) eta2 <- eta2 * 0.99
    }


    ## STEP 2: SAMPLE SIGMA2 GIVEN PHI & DATA

    # compute errors
    e  <- y - x * phi.draw

    # sum of squared residuals (equivalent to sum((e)^2))
    ee <- t(e) %*% e

    # compute posterior quantities
    ck <- c0 +  t/2
    dk <- d0 + ee/2
    # sample sigma2 from inverse gamma posterior
    sig2.draw <- 1 / rgamma(1, ck, dk)

    # STEP 3: STORE POSTERIOR SAMPLES

    #only after burn-in period!
    if(i > nburn){

    phi.store[i - nburn,]  <- phi.draw
    sig.store[i - nburn,] <- sig2.draw

    }


  }

  return(list(Phi = phi.store, Sig2 = sig.store))

}

# Estimate posteriors
yy <- un.demeaned[-1]
xx <- un.demeaned[-length(un.demeaned)]

priors <- c("norm", "norm", "beta", "beta", "beta")
prior.params <- list(c(0,1), c(1, 1), c(1, 1), c(10, 10), c(0.1, 0.1))

res <- list()

for (i in 1:length(priors)) {
  res[[i]] <- metro.est(yy, xx, priors[i], prior.params[[i]])
}
```

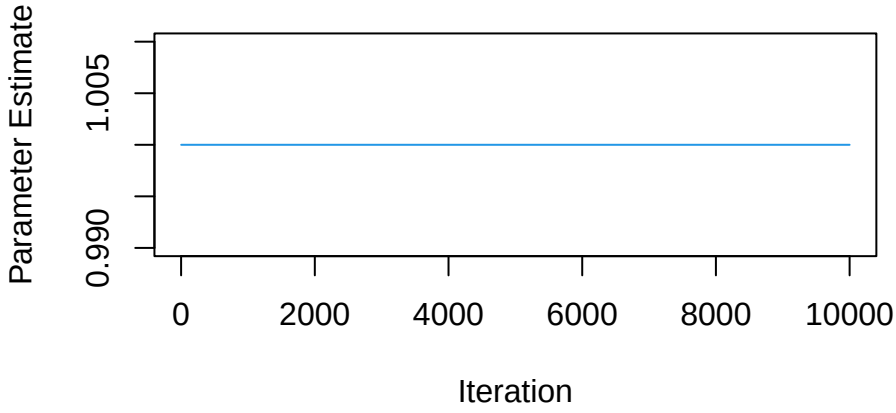
```r
# Traceplots and densities
invisible(lapply(res, function(x) plot.ts(x[[1]], xlab = "Iteration", ylab = "Parameter Estimate", col =
```



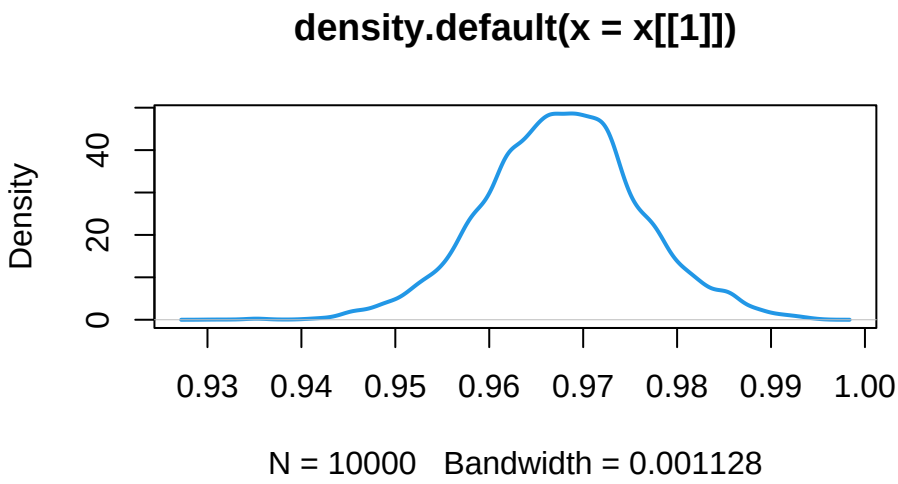


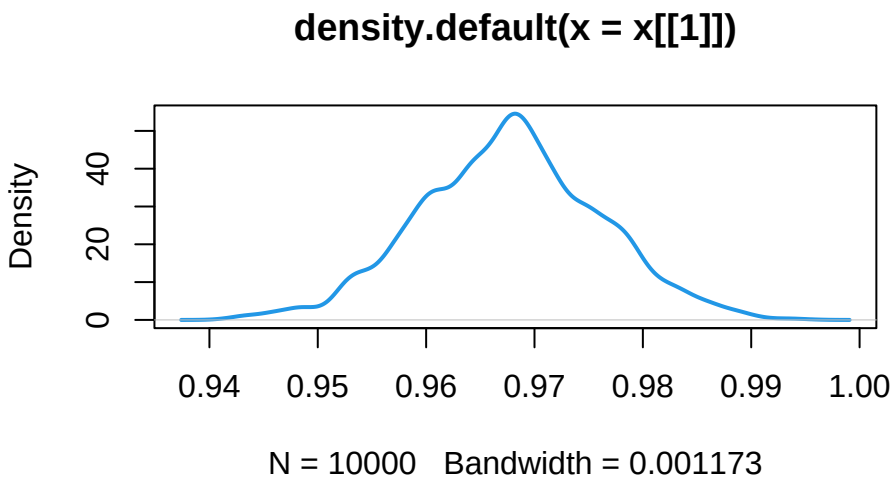

Parameter Estimate
0.93
0.96
0.99
0
2000
4000
6000
8000
10000
Iteration

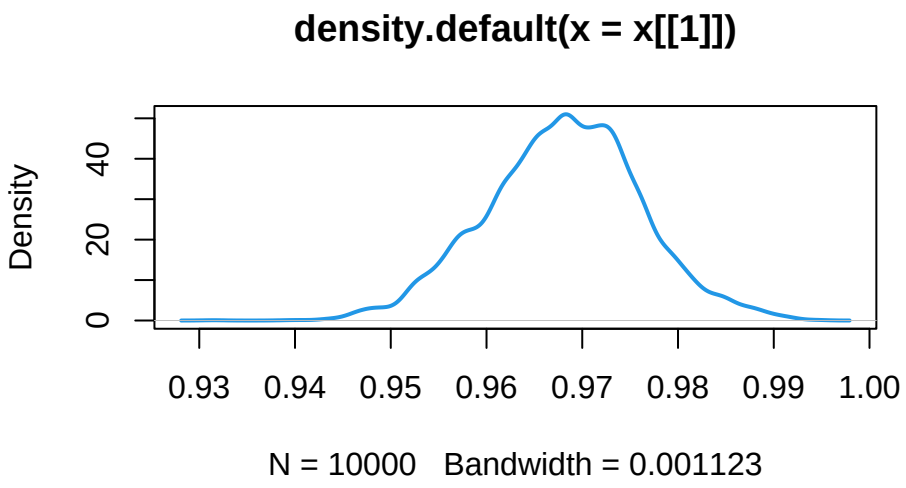

Parameter Estimate
0.94
0.97
0
2000
4000
6000
8000
10000
Iteration



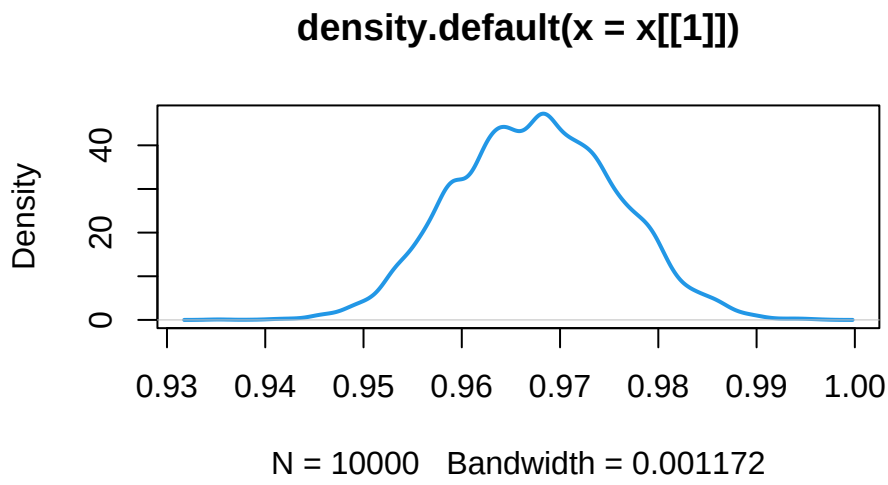

```
invisible(lapply(res, function(x) plot(density(x[[1]]), lwd = 2, col = 4)))
```

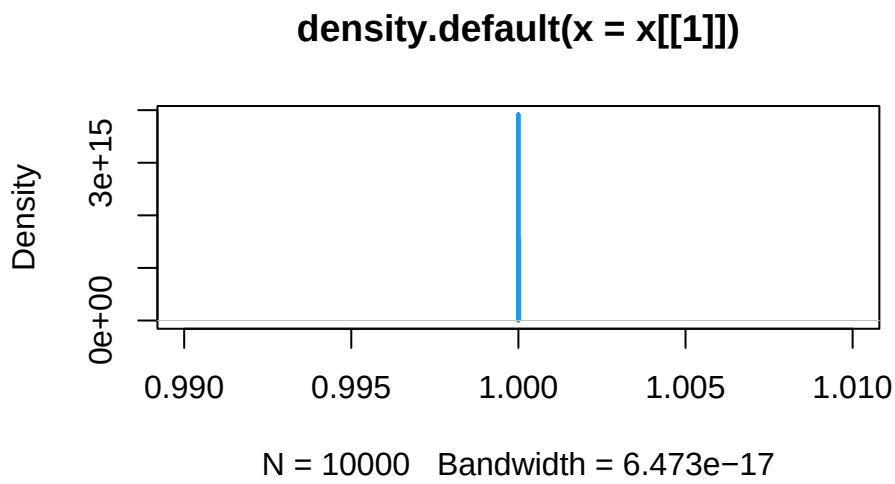






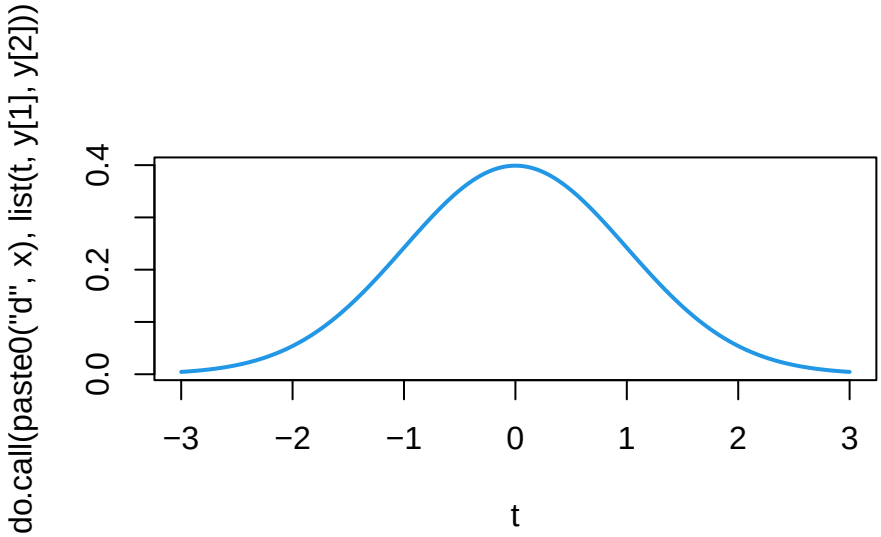

```
## Warning in regularize.values(x, y, ties, missing(ties), na.rm = na.rm):
## collapsing to unique 'x' values
```

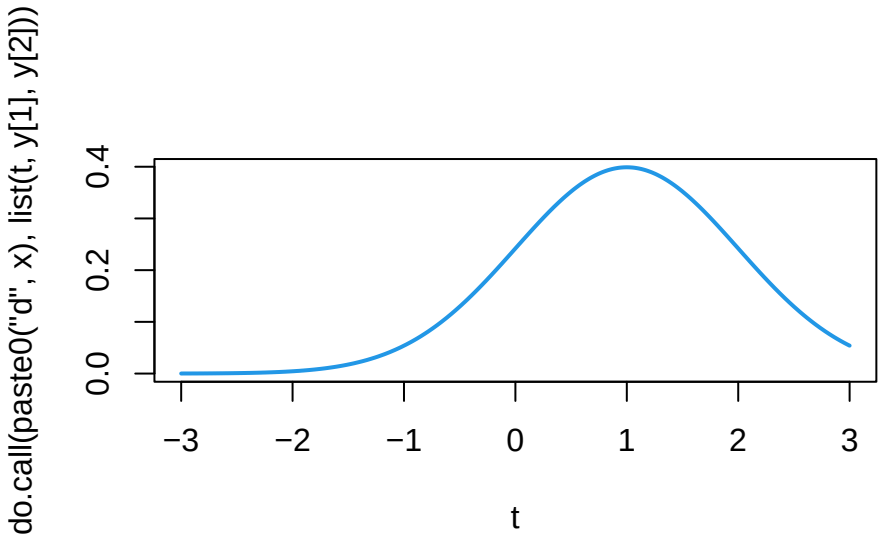


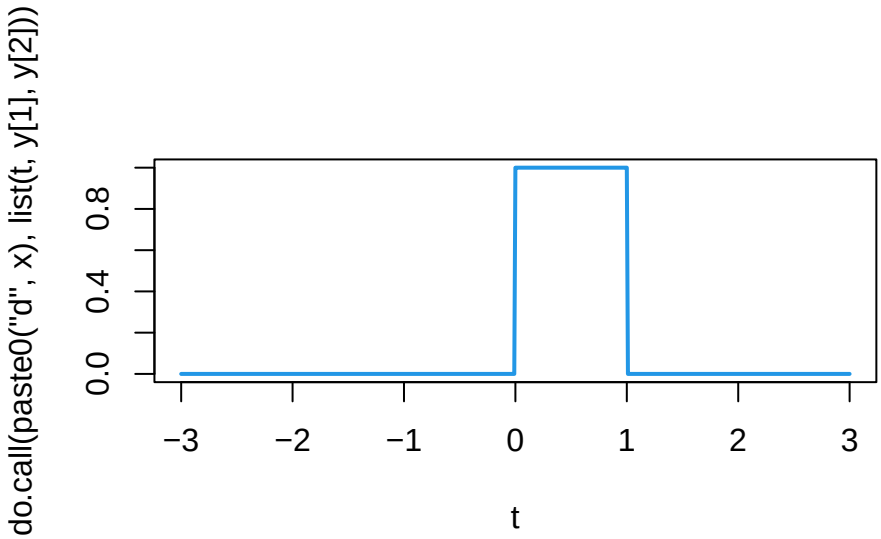


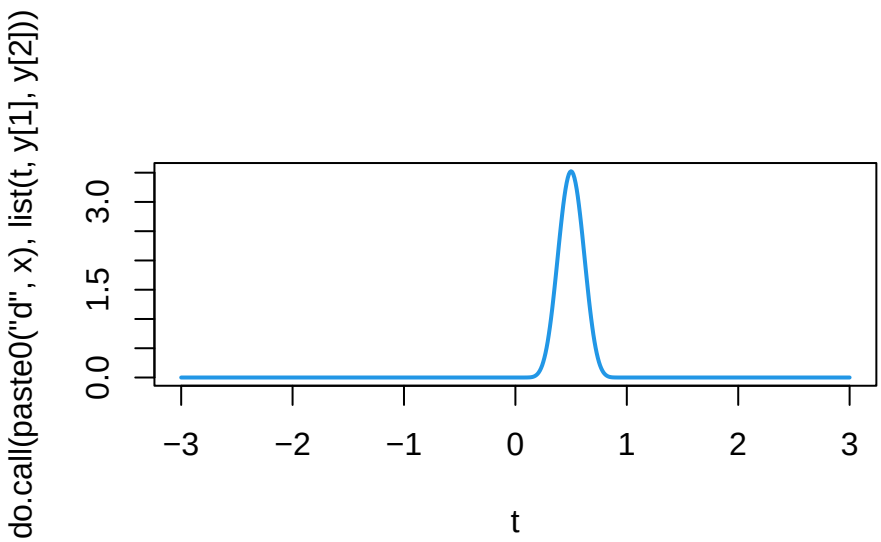
3

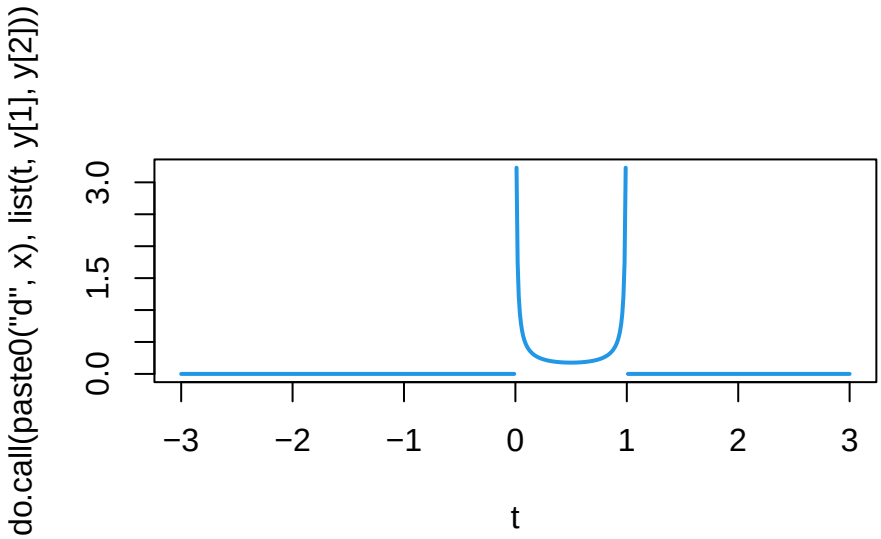
```
# Plot prior densities
invisible(Map(function(x, y){
  t <- seq(-3, 3, 0.01)
  plot(t, do.call(paste0("d", x), list(t, y[1], y[2])),
       type = "l", lwd = 2, col = 4)
}, priors, prior.params))
```


do.call(paste0("d", x), list(t, y[1], y[2]))
0.0
0.2
0.4
−3
−2
−1
0
1
2
3
t


do.call(paste0("d", x), list(t, y[1], y[2]))
0.0
0.2
0.4
−3
−2
−1
0
1
2
3
t


do.call(paste0("d", x), list(t, y[1], y[2]))
0.0
0.4
0.8
−3
−2
−1
0
1
2
3
t


do.call(paste0("d", x), list(t, y[1], y[2]))
0.0
1.5
3.0
−3
−2
−1
0
1
2
3
t


do.call(paste0("d", x), list(t, y[1], y[2]))
0.0
1.5
3.0
−3
−2
−1
0
1
2
3
t